

React `useContext` & `useEffect`

PART 1 — `useContext` Hook (Context API)

1 What is `useContext`?

`useContext` is a React hook used to **consume (read and update) data from a Context** created using `createContext`. It allows any component to access shared/global state **without prop drilling**.

2 Why `useContext` is Required

Problem: Prop Drilling

Passing `loginStatus` manually through multiple nested components is inefficient and hard to maintain.

Solution: Context API + `useContext`

- Centralizes shared state
- Makes data available globally
- Improves scalability

3 Flow of Context API

```
createContext()  
  ↓  
Context.Provider (value)  
  ↓  
useContext(Context)
```

4 Creating Context

```
import { createContext } from "react";  
export const LoginContext = createContext();
```

5 Providing Context Value

```
<LoginContext.Provider value={{ loginStatus, setLoginStatus }}>  
  <App />  
</LoginContext.Provider>
```

6 Consuming Context

```
const { loginStatus, setLoginStatus } = useContext(LoginContext);
```

7 Authentication Example

```
setLoginStatus(true);
```

```
<Route path="/home" element={loginStatus ? <Home /> : <Navigate to="/" />} />
```

8 Navbar Visibility

```
{loginStatus && <Navbar />}
```

9 Logout Logic

```
setLoginStatus(false);  
sessionStorage.clear();  
navigate("/");
```

Common Mistakes

- Importing `useContext` from `react-router-dom`
- Using array destructuring `[]` instead of `{}`
- Case mismatch in Context name
- Using Context without Provider

PART 2 — `useEffect` Hook

1 What is `useEffect`?

`useEffect` handles **side effects** like API calls, subscriptions, timers, and DOM updates.

2 Syntax

```
useEffect(() => {  
  // logic  
}, [dependencies]);
```

3 Execution Cases

- No dependency → runs on every render
- Empty array `[]` → runs once
- With dependencies → runs on change

4 Cleanup Function

```
useEffect(() => {  
  const id = setInterval(() => {}, 1000);  
  return () => clearInterval(id);  
}, []);
```

5 Lifecycle Mapping

Class Method	<code>useEffect</code>
<code>componentDidMount</code>	<code>useEffect(() => {}, [])</code>
<code>componentDidUpdate</code>	<code>useEffect(() => {}, [deps])</code>
<code>componentWillUnmount</code>	Cleanup

6 Profile Example

```
useEffect(() => {  
  getProfile();  
}, []);
```

7 Profile API Logic

```
const token = sessionStorage.getItem("token");
```

Common Mistakes

- Missing dependency array
 - Async function directly in effect
 - No cleanup
-

Summary

- `useContext` → Global state, avoids prop drilling
- `useEffect` → Lifecycle & side effects

SUNBEAM